

55. FormData

HTML forms bhejne ke liye (files ke saath ya bina, extra fields ke saath).

```
js  
  
let formData = new FormData([form]);
```

`form` element do to uske saare fields **apne aap** pakad leta hai.

Khaas baat: `fetch` jaise network methods `FormData` ko `body` ki tarah lete hain, aur wo `Content-Type: multipart/form-data` ke saath encode hoke jaata hai. Server ko lagta hai normal form submission hai.

Simple form bhejna (lagbhag one-liner)

```
html  
  
<form id="formElem">  
  <input type="text" name="name" value="John">  
  <input type="text" name="surname" value="Smith">  
  <input type="submit">  
</form>  
  
<script>  
formElem.onsubmit = async (e) => {  
  e.preventDefault();  
  
  let response = await fetch('/article/formdata/post/user', {  
    method: 'POST',  
    body: new FormData(formElem)  
  });  
  
  let result = await response.json();  
  alert(result.message);  
};  
</script>
```

FormData methods

- `append(name, value)`: field jodo
- `append(name, blob, fileName)`: aise jodo jaise `<input type="file">`. Teesra argument `fileName` **file ka naam** hai (field ka naam nahi).
- `delete(name)`

- `get(name)`
- `has(name): true/false`
- `set(name, value) / set(name, blob, fileName): append` jaisa, par pehle same naam ke saare fields hata deta hai, phir naya jodta hai (ek hi field pakka).

Ek form me same naam ke kai fields ho sakte hain, to `append` baar baar karne par sab jud jaate hain.

`for..of` se iterate:

```
js
for (let [name, value] of formData) alert(`${name} = ${value}`);
```

File ke saath form

Form hamesha `multipart/form-data` me jaata hai, jo files bhejne deta hai, to `<input type="file">` bhi normal form ki tarah chala jaata hai.

```
html
<form id="formElem">
  <input type="text" name="firstName" value="John">
  Picture: <input type="file" name="picture" accept="image/*">
  <input type="submit">
</form>
```

Code same: `body: new FormData(formElem)`.

Blob data ke saath form

Dynamic binary data (jaise canvas ki image) ko extra fields (naam waghera) ke saath ek form ki tarah bhejna aksar convenient hota hai, aur servers multipart forms ke liye zyaada taiyar hote hain.

```
js
```

```
let imageBlob = await new Promise(resolve => canvasElem.toBlob(resolve, 'image/

let formData = new FormData();
formData.append("firstName", "John");
formData.append("image", imageBlob, "image.png");

let response = await fetch('/article/formdata/post/image-form', {
  method: 'POST',
  body: formData
});
```

`formData.append("image", imageBlob, "image.png")` = jaise form me `<input type="file" name="image">` ho aur user ne `image.png` naam ki file chuni ho.

Summary

- `new FormData(form)` se HTML form pakdo, ya khaali `FormData` banake fields jodo.
- `set` same-name fields hata deta hai, `append` nahi. Bas yehi farak.
- File bhejne ke liye **3-argument syntax** chahiye (aakhri argument = file ka naam).
- Baaki: `delete`, `get`, `has`.